Lab 2 — Secrets Injection CICD and Segmentation Model Training

1. Dataset Description and Preprocessing

The dataset used for this project is the keremberke/satellite-building-segmentation dataset from HuggingFace, which contains aerial satellite images annotated with building footprints using bounding boxes. A total of 100 samples were initially collected and processed using the Week 7 pixel mask generation approach. During preprocessing, any samples whose masks contained no building pixels were discarded, resulting in 91 valid samples. These were then split into approximately 70/15/15 proportions, producing 63 training samples, 13 validation samples, and 15 test samples. For each image, the make_mask() function from the Week 7 notebook converts each bounding box [x_min, y_min, width, height] into a binary pixel mask by filling the corresponding rectangular region with ones. All masks for a given image are then merged using a logical OR operation to produce a single binary segmentation mask where pixel value 1 represents a building and 0 represents background. When the SAM ViT-H checkpoint is available, the preprocessing pipeline uses SamAutomaticMaskGenerator to generate candidate segments, and each segment is accepted if its IoU with any labelled bounding box exceeds 0.3, producing higher-quality building outlines. If SAM is unavailable, the pipeline automatically falls back to the Week 7 bounding-box mask generation method, ensuring that the preprocessing stage remains functional regardless of hardware constraints.

2. Model Architecture and Training Approach

The pretrained ResNet-50 image classifier from Lab 1 was replaced with the transformer-based semantic segmentation model SegFormer (nvidia/mit-b0). SegFormer uses a hierarchical Mix Transformer encoder combined with an MLP decoder head that fuses multi-scale features efficiently. The MiT-B0 variant was selected because it is computationally lightweight and suitable for CPU-based training while still benefiting from pretrained weights. The model was configured with a 2-class segmentation head, where class 0 represents background and class 1 represents house/building pixels. Training was performed for 5 epochs using a batch size of 2, learning rate of 6e-5, and input image size of 256 × 256. The optimizer used was AdamW with cosine annealing scheduling. The best checkpoint was selected based on the highest validation IoU. Across the five epochs, validation IoU improved from 0.4430 to 0.5634, while validation Dice score improved from 0.6023 to 0.7109. The best checkpoint was therefore saved at epoch 5 with validation IoU 0.5634 and validation Dice 0.7109.

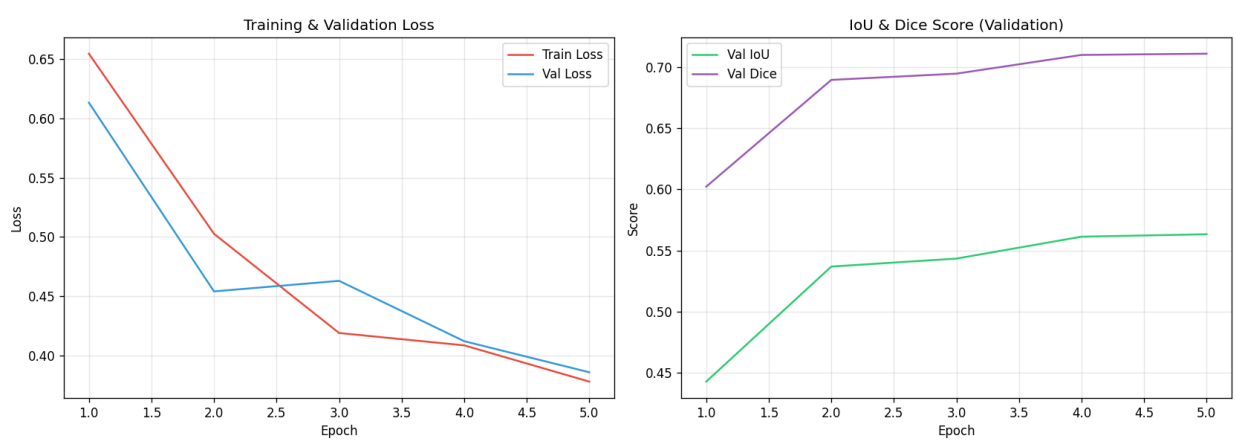

Figure 1. Training curves showing training loss, validation loss, IoU, and Dice over 5 epochs

3. Evaluation Metrics, Results, and Key Observations

The best checkpoint from epoch 5 was evaluated on the 15-sample held-out test set using evaluate.py. The two primary evaluation metrics used were Intersection over Union (IoU) and Dice score. IoU was computed as $(TP + \varepsilon) / (TP + FP + FN + \varepsilon)$ and Dice as $(2TP + \varepsilon) / (2TP + FP + FN + \varepsilon)$, where $\varepsilon = 1e\text{-}6$ was used for numerical stability. The model achieved a mean test IoU of 0.4589 with standard deviation ±0.2212, and a mean Dice score of 0.5934 with standard deviation ±0.2408. These results are reasonable considering the small dataset size and the fact that the segmentation masks were partially derived from bounding-box approximations. The training and validation loss curves decreased consistently across all five epochs, suggesting that no clear overfitting was observed within the training budget. The relatively high standard deviation reflects the diversity of aerial scenes in the test set, as some images contain dense building clusters while others contain sparse or partially occluded structures that are harder to segment accurately.

4. Issues Encountered and Solutions

Several issues were encountered during the project. The most significant challenge was class imbalance, as aerial images contain substantially more background pixels than building pixels. To reduce the likelihood of the model learning an all-background shortcut, samples with zero positive pixels were discarded during preprocessing. Another issue was the limited dataset size, as the 100-sample cap was chosen to keep preprocessing and training feasible on CPU. While this restricts model generalization, the use of transfer learning through SegFormer helps improve performance despite the small dataset. Another challenge was the SAM ViT-H checkpoint requirement, which is a large 2.4 GB file. To ensure robustness, the preprocessing pipeline includes an automatic fallback to the Week 7 make_mask() method whenever SAM is unavailable. Finally, SegFormer outputs logits at one-quarter of the input resolution, meaning 64 × 64 logits for 256 × 256 images. To address this, ground-truth masks were downsampled during training and predictions were upsampled during evaluation using bilinear interpolation.

5. Visualisations of Sample Predictions

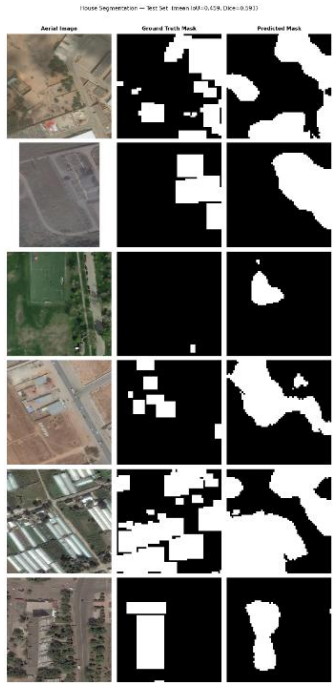

Figure 2. Sample predictions from the test set showing aerial image, ground-truth mask, and predicted mask.

The figure above shows 6 test-set samples. White pixels indicate regions classified as house/building. The model correctly identifies large building footprints and contiguous rooftop regions. Smaller or partially occluded structures, and buildings at image edges, are harder to delineate accurately, which accounts for the variance in per-sample IoU scores.

6. Secrets Injection

Sensitive configuration values such as model checkpoint paths, runtime hyperparameters, HuggingFace tokens, and Docker Hub credentials are managed securely using environment variables. The project uses python-dotenv to load variables from a .env file at runtime, and the .env file is excluded from version control through .gitignore. Docker Hub credentials are stored as encrypted GitHub repository secrets (DOCKERHUB_USERNAME and DOCKERHUB_TOKEN) and are never hard-coded anywhere in the codebase. This ensures secure handling of sensitive information while allowing the application to remain fully configurable at runtime.

7. CI/CD Pipeline

A GitHub Actions CI/CD pipeline was implemented to automate testing, Docker image building, and deployment. The workflow runs on every push to main or develop and on pull requests to main. The first stage installs CPU-only PyTorch, installs all project dependencies, and runs 10 automated pytest unit tests with coverage reporting. The second stage runs only on direct pushes to main after all tests pass. This stage builds the Docker image using Docker Buildx, tags the image as both latest and a short SHA identifier, and pushes it to Docker Hub under the repository dvdmalek/house-segmentation-service.

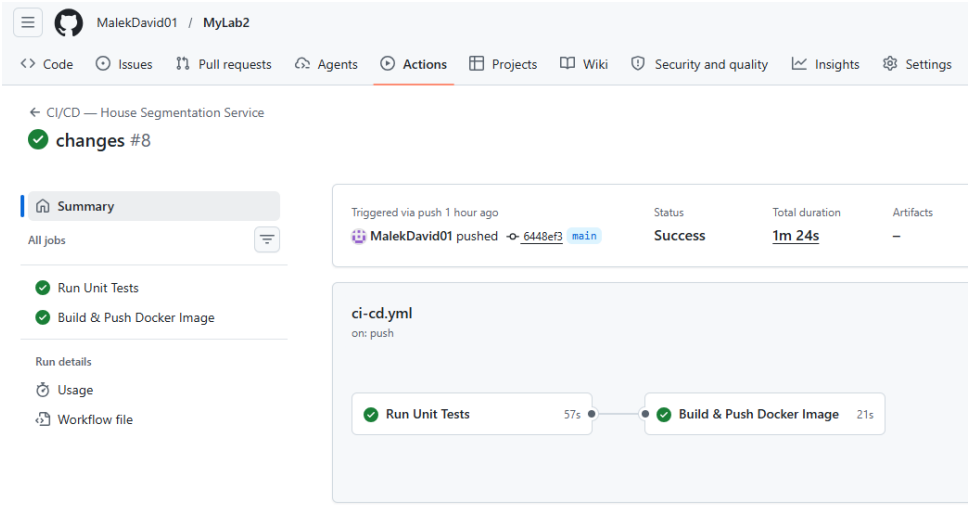

Figure 3. GitHub Actions screenshot showing the full pipeline run with both jobs (Run Unit Tests and Build & Push Docker Image) with green checkmarks

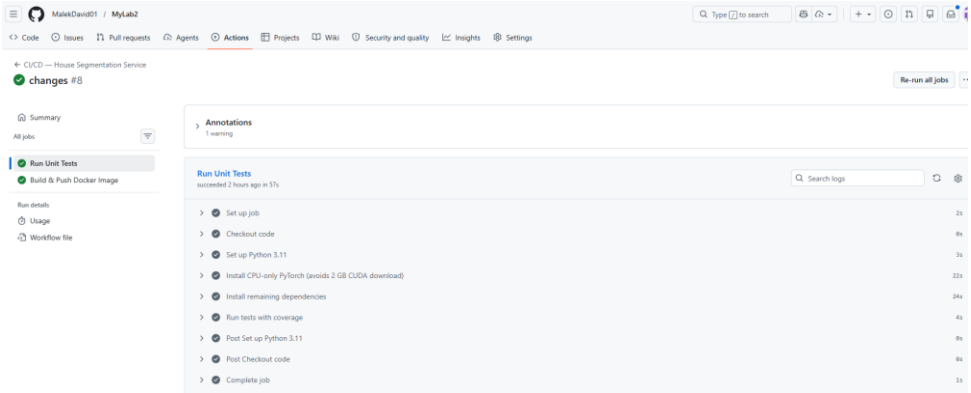

Figure 4. GitHub Actions screenshot zoomed into the Run Unit Tests job showing all 10 pytest tests passing

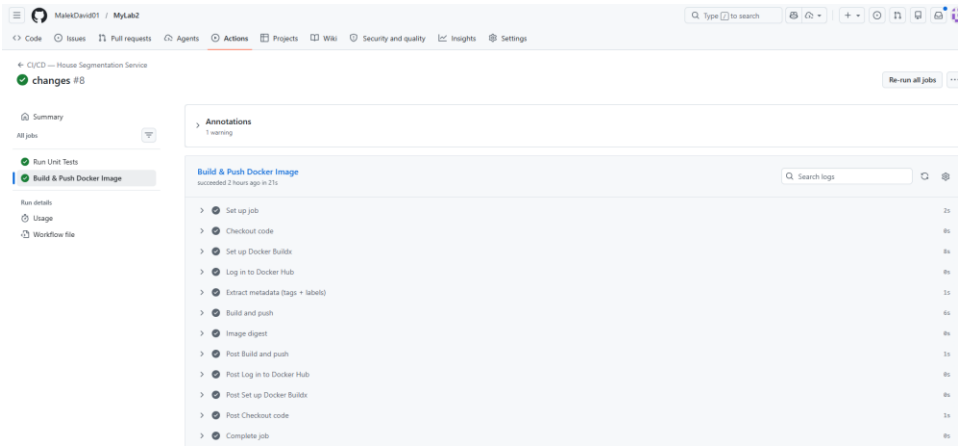

Figure 5. GitHub Actions screenshot zoomed into the Build & Push Docker Image job showing the push step completing successfully

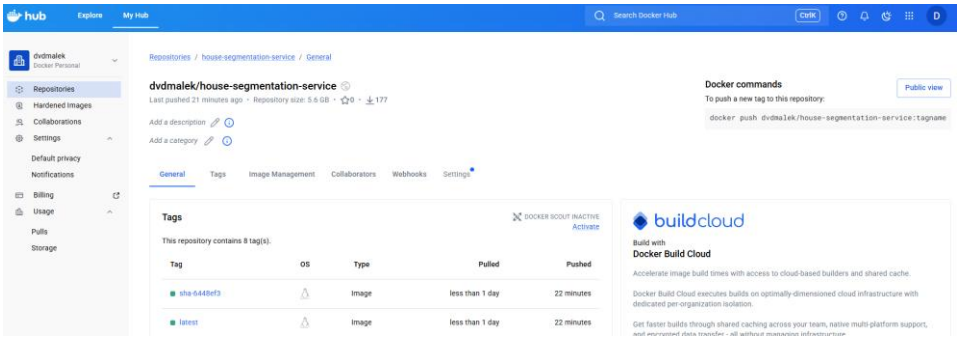

Figure 6. Docker Hub page showing dvdmalek/house-segmentation-service with the latest tag and push timestamp